

Micro Hack 2 - SDC Workloads Workbook

FY27 - Fabric Apps Motion (EMEA)

Contents

1	Micro Hack 2 — SDC Workloads Workbook (Participants)	1
1.1	Day agenda (1 day)	1
2	Part A — Understand	2
2.1	1) The scenario in plain words	2
2.2	2) The three pieces (and who owns the algorithm)	4
2.3	3) Key words (30-second primer)	4
2.4	4) Team framing (fill this in 2 minutes, then build)	4
3	Part B — Build it step by step (full solution)	4
3.1	Step 1 — Check the SaaS is alive (1 min)	5
3.2	Step 2 — Create the empty item	5
3.3	Step 3 — Describe the data shape (the contract)	5
3.4	Step 4 — Call the SaaS and render with fake data	6
3.5	Step 5 — Switch the source to real OneLake data	8
3.6	Step 6 — Make it graphical (polish)	10
3.7	Step 7 — Run it inside Fabric (Dev Gateway)	10
4	Wrap-up	11
4.1	Demo storyboard (5 minutes)	11
4.2	Reflection	11
4.3	Reference assets	11

1 Micro Hack 2 — SDC Workloads Workbook (Participants)

Track: **SDC Workloads (Fabric Extensibility Toolkit)** Scenario: **GreenGrid Analytics** (fictional SDC)

Print-ready PDF: [microhack-2-sdc-workloads-workbook.pdf](#)

This is a **didactic** micro-hack. You are **not** expected to *find* the solution — every step explains **what** you do, **why**, and **what you should see**, and gives you the exact code to paste. Follow them in order and you get a working Fabric workload.

1.1 Day agenda (1 day)

Morning — presentations & demos. Afternoon — hands-on hack. This is a **standalone one-day micro-hack** for the SDC Workloads (Extensibility Toolkit) track.

Time	Session
09:00	Welcome & motion overview — the Fabric apps motion for EMEA
09:30	Microsoft Fabric foundations — OneLake, capacity, workspaces
10:00	Extensibility Toolkit foundations — workloads, items, Dev Gateway
10:30	Break
10:45	Demo · GreenGrid Scorecard — the workload you'll build + the SaaS prerequisite
11:15	The hack brief, teams & environment check
12:00	Lunch
13:30	Hack · Sprint 1 — scaffold & run (steps 1–4)
15:30	Break
15:45	Hack · Sprint 2 — integrate & polish (steps 5–7)
16:45	Team demos (5 min each)
17:00	Wrap-up, KPIs & next steps

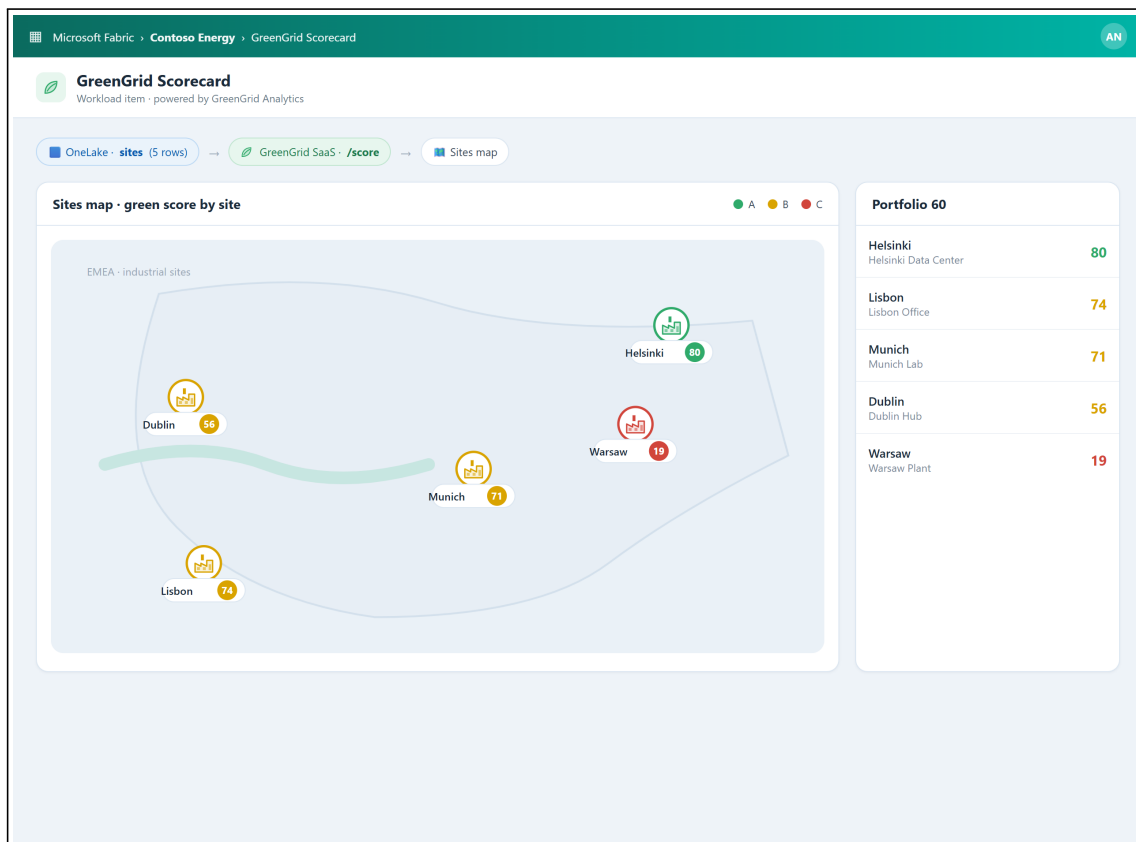
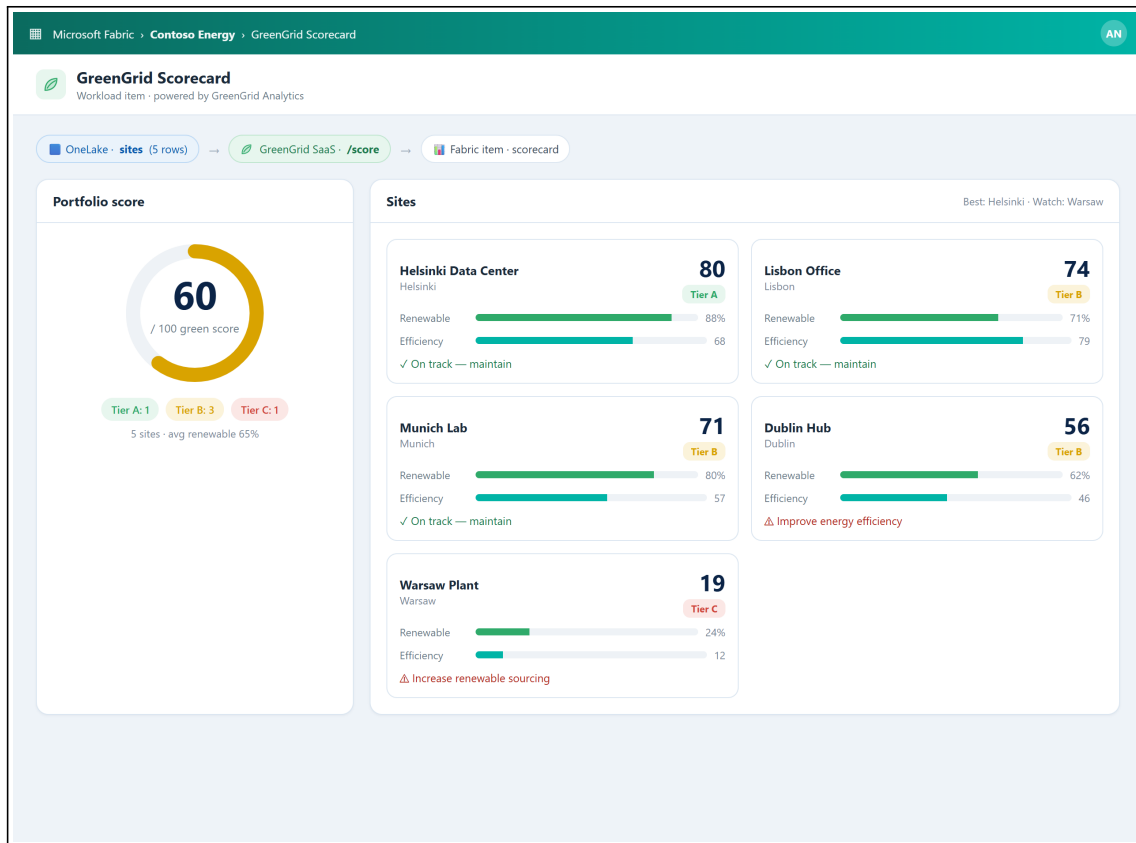
The Business Apps (Rayfin) track is a **separate** one-day micro-hack with its own work-book.

2 Part A — Understand

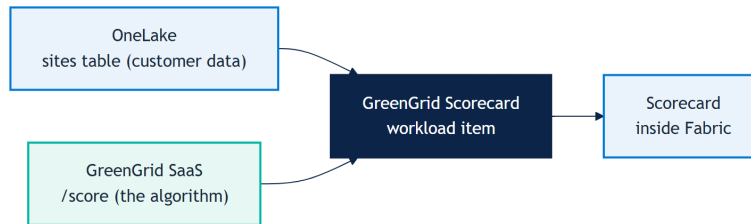
2.1 1) The scenario in plain words

GreenGrid Analytics is a software company (an **SDC** — a solution development company). Its product scores how **sustainable** a customer's sites are, from their energy use and renewable mix. The customer's data already lives in Microsoft Fabric (**OneLake**). Your job: bring GreenGrid's scoring **inside Fabric**, as a native screen, **without moving the data**.

The result looks like this — a sustainability scorecard and a map of the industrial sites, rendered inside Fabric:



2.2 2) The three pieces (and who owns the algorithm)



- **OneLake** = where the **customer's data** lives (a sites table).
- **GreenGrid SaaS** = a small web service that **holds the scoring algorithm** (GreenGrid's IP). You send it sites, it returns scores. *The trainer deploys it for you — it is a prerequisite.*
- **The workload** = the piece **you build today**: a Fabric *item* that reads OneLake, calls the SaaS, and draws the scorecard.

2.3 3) Key words (30-second primer)

- **Workload**: a web app that runs *inside* the Fabric portal (in an iFrame). You host it, Fabric displays it.
- **Item**: one thing a user creates in a workspace (here: a *GreenGrid Scorecard*). Like a Notebook or a Lakehouse, but yours.
- **Extensibility Toolkit**: Microsoft's starter kit + SDK to build such workloads quickly.
- **OBO token** (On-Behalf-Of): a secure token the workload gets so it can read the user's OneLake data **as that user** — no passwords, no data copies.
- **Dev Gateway**: a local bridge that lets Fabric show your workload while it runs on **your machine** — perfect for a hack.

2.4 4) Team framing (fill this in 2 minutes, then build)

- Team name:
- SaaS URL + API key (from the trainer):
- One sentence you'll say in the demo:

3 Part B — Build it step by step (full solution)

Golden rule (read this!): make the flow work with **seed (fake) data first** (steps 1–4), *then* switch the source to real OneLake data (step 5). Don't fight OneLake security before anything appears on screen — you'll waste the afternoon.

Each step tells you **what it is**, **why**, **where to paste** and **what you should see**. Copy the code as-is.

3.1 Step 1 — Check the SaaS is alive (1 min)

What this is. A quick test that the GreenGrid service (the algorithm) is reachable. **Why.** Your workload will call it; if it's down, nothing scores. Test it before coding.

```
curl -i <SAAS_URL>/health          # expect: 200
curl -s -X POST <SAAS_URL>/score \
  -H "Content-Type: application/json" -H "x-api-key: <API_KEY>" \
  -d '{"sites":[{"siteId":"s1","name":"Helsinki
    ↪ DC","city":"Helsinki","energyKwh":320,"renewablePct":88}]}'
```

**** What you should see.**** /health returns 200; /score returns a greenScore and a tier. (Local fallback: `cd src/workloadsdc && npm install && npm run saas:start`, URL `http://localhost:8787`, key `greengrid-demo-key`.)

3.2 Step 2 — Create the empty item

What this is. Generate a new Fabric *item type* from the Starter-Kit, using GitHub Copilot. **Why.** The item is the container your scorecard lives in. Start empty, fill it next.

Create a new Extensibility Toolkit item type "GreenGridScorecard" with an
 ↪ editor tab and a
 Leaf icon. Keep the default item creation flow.

**** What you should see.**** A new item folder (e.g. `app/items/GreenGridScorecard/`) and an empty editor that opens without errors.

3.3 Step 3 — Describe the data shape (the contract)

What this is. TypeScript types that describe what you send to the SaaS and what it returns. **Why.** A shared “contract” means your code and the SaaS agree on field names — fewer bugs.

Create `app/items/GreenGridScorecard/contracts.ts` and paste:

```
export type SiteRecord = {
  siteId: string; name: string; city: string; energyKwh: number;
  ↪ renewablePct: number;
};
export type GreenTier = 'A' | 'B' | 'C';
export type ScoredSite = SiteRecord & {
  efficiency: number; greenScore: number; tier: GreenTier; tip: string;
};
export type ScoreResponse = {
  sites: ScoredSite[];
  summary: {
    totalSites: number; avgScore: number; avgRenewablePct: number;
    tierCounts: Record<GreenTier, number>; best: ScoredSite; worst:
    ↪ ScoredSite;
```

```
};
};
```

**** What you should see.** The file compiles (no red squiggles) and can be imported.

3.4 Step 4 — Call the SaaS and render with fake data

This is the big one: it makes the **end-to-end flow visible**. Three small files.

3.4.1 4a. The SaaS client — greengridClient.ts

What this is. A function that POSTs sites to the SaaS and returns the scores.

```
import type { ScoreResponse, SiteRecord } from './contracts';

const SAAS_URL = 'http://localhost:8787';           // <- replace with the
  ↳ trainer URL
const API_KEY = 'greengrid-demo-key';              // <- replace with the
  ↳ trainer key

export async function scorePortfolio(sites: SiteRecord[]):
  ↳ Promise<ScoreResponse> {
  const res = await fetch(`${SAAS_URL}/score`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'x-api-key': API_KEY },
    body: JSON.stringify({ sites }),
  });
  if (!res.ok) throw new Error(`GreenGrid SaaS error ${res.status}`);
  return res.json();
}
```

3.4.2 4b. Fake data — seed.ts

What this is. A handful of sample sites so you can see results before touching OneLake.

```
import type { SiteRecord } from './contracts';
export const seedSites: SiteRecord[] = [
  { siteId: 'site-hel', name: 'Helsinki Data Center', city: 'Helsinki',
    ↳ energyKwh: 320, renewablePct: 88 },
  { siteId: 'site-lis', name: 'Lisbon Office', city: 'Lisbon', energyKwh:
    ↳ 210, renewablePct: 71 },
  { siteId: 'site-muc', name: 'Munich Lab', city: 'Munich', energyKwh:
    ↳ 430, renewablePct: 80 },
  { siteId: 'site-dub', name: 'Dublin Hub', city: 'Dublin', energyKwh:
    ↳ 540, renewablePct: 62 },
  { siteId: 'site-waw', name: 'Warsaw Plant', city: 'Warsaw', energyKwh:
    ↳ 880, renewablePct: 24 },
]
```

```
];
```

3.4.3 4c. The screen — Editor.tsx

What this is. The React component that calls the SaaS on open and shows the scorecard.

```
import React, { useEffect, useState } from 'react';
import { FluentProvider, webLightTheme, Spinner, Badge } from
  ↪ '@fluentui/react-components';
import { scorePortfolio } from './greengridClient';
import { seedSites } from './seed';
import type { ScoreResponse } from './contracts';

const tierColor = (t: 'A' | 'B' | 'C') => (t === 'A' ? '#2faa6a' : t ===
  ↪ 'B' ? '#d9a300' : '#d1453b');

export default function Editor() {
  const [data, setData] = useState<ScoreResponse | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // Step 4: fake data. In Step 5 you replace `seedSites` with OneLake
    ↪ rows.
    scorePortfolio(seedSites).then(setData).catch((e) =>
    ↪ setError(String(e)));
  }, []);

  if (error) return <div role="alert">Failed: {error}</div>;
  if (!data) return <Spinner label="Scoring sites..." />;

  return (
    <FluentProvider theme={webLightTheme}>
      <div style={{ padding: 24 }}>
        <h1>GreenGrid Scorecard</h1>
        <p>Data from OneLake, scored by GreenGrid</p>
        <div style={{ fontSize: 48, fontWeight: 700
    ↪ }}>{data.summary.avgScore}<span style={{ fontSize: 18 }}> /
    ↪ 100</span></div>
        <div style={{ display: 'grid', gridTemplateColumns:
    ↪ 'repeat(2,1fr)', gap: 12, marginTop: 16 }}>
          {data.sites.map((s) => (
            <div key={s.siteId} style={{ border: '1px solid #dbe6f1',
    ↪ borderRadius: 12, padding: 14 }}>
              <div style={{ display: 'flex', justifyContent:
    ↪ 'space-between' }}>
                <strong>{s.name}</strong>
                <Badge appearance="filled" style={{ background:
    ↪ tierColor(s.tier) }}>Tier {s.tier}</Badge>
```



```

        </div>
        <div style={{ color: '#7a8a9a', fontSize: 12 }}>{s.city}</div>
        <div style={{ fontSize: 26, fontWeight: 700
↵  }}>{s.greenScore}</div>
        <div style={{ fontSize: 13 }}>{s.tip}</div>
      </div>
    )}}
  </div>
</div>
</FluentProvider>
);
}

```

**** What you should see — Milestone M1 (Sprint 1). **** The item shows the portfolio score (a big number) and 5 site cards with their tier. The **whole chain works**: your screen -> the SaaS -> back to the screen. This alone is demoable.

3.5 Step 5 — Switch the source to real OneLake data

What this is. Replace the fake seedSites with the customer's **real** site data stored in OneLake.

3.5.1 Where does the data live? (read this first)

In Fabric, a customer's data lives in a **Lakehouse** inside **OneLake**. A Lakehouse has two areas:

- **Files** — raw files: CSV, Parquet, JSON...
- **Tables** — managed Delta tables (queryable, governed).

For this hack the customer sites are provided as **one CSV file** in the Lakehouse: `Files/sites.csv`. (A CSV is the simplest thing to read from a workload. A Delta Tables/sites would also work, but reading Delta needs a SQL endpoint or a backend — out of scope for one day.)

Format of Files/sites.csv — a header row, then one line per site:

```

siteId,name,city,energyKwh,renewablePct
site-hel,Helsinki Data Center,Helsinki,320,88
site-lis,Lisbon Office,Lisbon,210,71
site-muc,Munich Lab,Munich,430,80
site-dub,Dublin Hub,Dublin,540,62
site-waw,Warsaw Plant,Warsaw,880,24

```

The trainer pre-loads this file (in Fabric: open the Lakehouse -> **Files** -> **Upload**). A ready-made copy is in the kit at `src/workloadsrc/data/sites.csv`. To test your own data, upload a CSV with the **same header** to Files/.

You also need two IDs to build the OneLake path (both appear in the URL when the Lakehouse is open in Fabric, and are available from the item context):

- `workspaceId` — the workspace GUID.

- lakehouseId — the Lakehouse GUID.

3.5.2 5a. The OneLake reader — onelake.ts

This reads the CSV file from OneLake (as the signed-in user, via an OBO token) and parses it.

```
import type { WorkloadClientAPI } from '@ms-fabric/workload-client';
import type { SiteRecord } from './contracts';

export async function readSites(
  client: WorkloadClientAPI, workspaceId: string, lakehouseId: string
): Promise<SiteRecord[]> {
  // 1) Ask Fabric for a token to read OneLake AS the signed-in user (OBO).
  const { token } = await client.auth.acquireAccessToken({
    additionalScopesToConsent:
    ↪ ['https://storage.azure.com/user_impersonation'],
  });
  // 2) Read the CSV file from the Lakehouse "Files" area.
  const url = `https://onelake.dfs.fabric.microsoft.com/${workspaceId}/${lakehouseId}/Files/sites.csv`;
  ↪ lakehouseId};
  const res = await fetch(url, { headers: { Authorization: `Bearer ${token}` } });
  ↪ ${token}` } });
  if (!res.ok) throw new Error(`OneLake read failed: ${res.status}`);
  // 3) Parse the CSV text into SiteRecord rows.
  return parseSitesCsv(await res.text());
}

function parseSitesCsv(csv: string): SiteRecord[] {
  const [header, ...lines] = csv.trim().split(/\r?\n/);
  const cols = header.split(',').map((c) => c.trim());
  return lines.filter(Boolean).map((line) => {
    const cells = line.split(',');
    const row = Object.fromEntries(cols.map((c, i) => [c, (cells[i] ?? '').trim()]));
    ↪ '').trim()]));
    return {
      siteId: row.siteId, name: row.name, city: row.city,
      energyKwh: Number(row.energyKwh), renewablePct:
    ↪ Number(row.renewablePct),
    };
  });
}
```

Note: this is a minimal CSV parser (split on commas). It's fine for this dataset; for names containing commas you'd use a real CSV library.

3.5.3 5b. Use it in Editor.tsx

Swap one line in the useEffect: read OneLake, then score.

```
// before (fake data):
scorePortfolio(seedSites).then(setData).catch((e) => setError(String(e)));

// after (real OneLake data; props: workloadClient, workspaceId,
  ↳ lakehouseId):
readSites(workloadClient, workspaceId, lakehouseId)
  .then(scorePortfolio)
  .then(setData)
  .catch((e) => setError(String(e)));
```

**** What you should see — Milestone M2 (Sprint 2).**** The same scorecard, now built from the **OneLake CSV**. (If the OBO token gives you trouble, keep `seedSites` as a fallback and keep moving — don't lose time on auth.)

3.6 Step 6 — Make it graphical (polish)

What this is. Upgrade the plain list into the polished scorecard + map. **Why.** A graphical result is what sells the story in the demo.

Ask Copilot (keep the data flow from steps 4–5):

```
Upgrade GreenGrid Scorecard: add a circular green-score gauge for the
  ↳ portfolio,
an A/B/C tier distribution, provenance chips "Data from OneLake" then
  ↳ "Scored by GreenGrid",
and a second view showing the sites as factory markers on a simple map,
  ↳ colored by tier.
Style: Fluent UI, teal #00b4a6 + green accents.
```

**** What you should see — Milestone M3 (Sprint 2).**** A gauge, A/B/C tiers, and the sites map — matching the example images at the top.

3.7 Step 7 — Run it inside Fabric (Dev Gateway)

What this is. Show your locally-running workload **inside the Fabric portal**. **Why.** That's how a workload really looks to a user — and how you demo it.

```
npm run start                # frontend dev server (serves the item UI)
npm run start:devGateway     # registers the dev workload with Fabric
```

Then in Fabric: **Settings -> Developer settings -> enable Developer mode**, open your workspace, then **+ New -> GreenGridScorecard**.

**** What you should see.**** The `GreenGridScorecard` item appears in Fabric and renders your scorecard, served from your machine. You're ready to demo. (Not listed? Developer mode is off, or the Dev Gateway isn't running.)

4 Wrap-up

4.1 Demo storyboard (5 minutes)

1. **Context** (30s): the SaaS value (the algorithm) you brought into Fabric.
2. **Flow** (3m): open the item, read OneLake, call the SaaS, show the scorecard + map.
3. **Architecture** (45s): OneLake (data) + SaaS (algorithm) + workload (native UI).
4. **Publish plan** (45s): from Dev Gateway to a released workload.

4.2 Reflection

- Where did the OneLake -> SaaS contract surprise you?
- What stays in the SaaS (the IP) vs. in the workload (the experience)?
- What would you harden before production?

4.3 Reference assets

- Full solution code: `src/workloadsrc/`
- SaaS prerequisite: `src/workloadsrc/saas/`
- Canonical spec: `src/workloadsrc/src/specs/greengrid-workload-spec.md`
- Trainer answer key + screenshots: `docs/microhack-2-sdc-workloads-solution.md`